Chapitre 1 — Introduction aux bases de données relationnelles

Une base de données relationnelle range les informations dans des tables. Chaque table représente un type d'objet, par exemple des étudiants, des cours ou des inscriptions. Une ligne correspond à un objet précis et une colonne à une de ses propriétés. Ce modèle a été proposé par Edgar F. Codd en 1970 et il reste aujourd'hui le plus utilisé en entreprise, car il est simple à comprendre et repose sur des bases mathématiques solides.

Chaque table possède une clé primaire : une colonne, ou un groupe de colonnes, dont la valeur est unique pour chaque ligne. Elle permet d'identifier une ligne sans ambiguïté. Une clé étrangère est une colonne qui fait référence à la clé primaire d'une autre table. C'est elle qui crée les liens entre les tables, par exemple entre une inscription et l'étudiant concerné.

Le langage SQL (Structured Query Language) sert à décrire les données que l'on veut obtenir, et non la manière de les obtenir. C'est le moteur de la base qui choisit le meilleur chemin pour exécuter la requête. On distingue plusieurs familles d'instructions : la définition des tables avec CREATE et ALTER, la manipulation des données avec INSERT, UPDATE et DELETE, et l'interrogation avec SELECT.

À retenir : une bonne base commence par un bon schéma. Avant d'écrire la moindre requête, on identifie les entités, leurs attributs et les relations entre elles.

Chapitre 2 — Requêtes de sélection

L'instruction SELECT permet de lire des données. Sa forme la plus simple indique les colonnes voulues et la table source : SELECT nom, prenom FROM etudiants. L'étoile permet de récupérer toutes les colonnes, mais on l'évite en production car elle renvoie souvent plus de données que nécessaire et rend le code fragile si la table évolue.

La clause WHERE filtre les lignes selon une condition. On peut combiner plusieurs conditions avec AND et OR, et utiliser des opérateurs comme BETWEEN, IN ou LIKE. Par exemple, WHERE ville IN ('Paris', 'Nice') garde seulement les étudiants de ces deux villes. Attention à la valeur NULL : elle ne se compare pas avec le signe égal, il faut écrire IS NULL ou IS NOT NULL.

La clause ORDER BY trie le résultat, par ordre croissant par défaut ou décroissant avec DESC. LIMIT restreint le nombre de lignes renvoyées, ce qui est utile pour afficher les dix premiers résultats d'un classement. DISTINCT supprime les doublons.

L'ordre d'écriture d'une requête n'est pas l'ordre d'exécution. Le moteur lit d'abord FROM, puis WHERE, puis SELECT, puis ORDER BY. Comprendre cet ordre explique pourquoi un alias défini dans SELECT ne peut pas être utilisé dans WHERE.

Chapitre 3 — Jointures

Les jointures combinent les lignes de plusieurs tables à partir d'une condition, en général l'égalité entre une clé étrangère et une clé primaire. La jointure interne, INNER JOIN, ne garde que les lignes qui ont une correspondance des deux côtés. Si un étudiant n'est inscrit à aucun cours, il n'apparaît pas dans le résultat.

La jointure externe gauche, LEFT JOIN, garde toutes les lignes de la table de gauche, même sans correspondance. Les colonnes de la table de droite valent alors NULL. C'est la jointure à utiliser pour répondre à une question comme « quels étudiants n'ont rendu aucun devoir ? » : on fait un LEFT JOIN puis on filtre les lignes où la clé de droite est NULL.

Il existe aussi RIGHT JOIN, symétrique de LEFT JOIN, et FULL OUTER JOIN, qui garde toutes les lignes des deux côtés. SQLite ne propose pas toutes ces variantes dans ses anciennes versions, ce qui oblige parfois à réécrire la requête avec un LEFT JOIN.

Une erreur fréquente consiste à oublier la condition de jointure. On obtient alors un produit cartésien : chaque ligne de la première table est associée à chaque ligne de la seconde, et le résultat explose en taille.

Chapitre 4 — Agrégations et regroupements

Les fonctions d'agrégation calculent une valeur à partir de plusieurs lignes : COUNT compte, SUM additionne, AVG calcule une moyenne, MIN et MAX donnent les extrêmes. Utilisées seules, elles résument toute la table en une seule ligne.

La clause GROUP BY forme des groupes de lignes qui partagent la même valeur, puis applique l'agrégation à chaque groupe. Par exemple, on peut compter le nombre d'inscrits par cours. Toute colonne du SELECT qui n'est pas agrégée doit apparaître dans le GROUP BY.

Pour filtrer sur le résultat d'une agrégation, on utilise HAVING et non WHERE, car WHERE agit avant le regroupement. Exemple : garder seulement les cours qui ont plus de trente inscrits.

Chapitre 5 — Normalisation

La normalisation est une méthode pour organiser les tables afin d'éviter la redondance et les incohérences. Une information ne doit être stockée qu'à un seul endroit : si l'adresse d'un étudiant est recopiée dans chaque inscription, il suffit d'oublier une mise à jour pour que la base se contredise.

La première forme normale impose des valeurs atomiques : une cellule ne contient qu'une seule valeur. La deuxième forme normale impose que chaque colonne dépende de toute la clé primaire, et pas seulement d'une partie. La troisième forme normale interdit qu'une colonne
dépende d'une autre colonne qui n'est pas une clé.

En pratique, on normalise jusqu'à la troisième forme normale, puis on dénormalise parfois volontairement pour gagner en performance, par exemple dans un entrepôt de données.